PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 1
3. Product Purpose 2
4. System Functionalities 2
5. User Stories & Use Cases 3
6. Features Included (Scope Definition) 3
7. Features Not Included (Scope Control) 3
8. Assumptions & Constraints 4
9. Risks & Questions Throughout Development 4

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: GridGuard

Version: 1.0

1. Project Overview

- **Problem Statement:** Municipal governments are responsible for maintaining public infrastructure across sprawling urban environments such as roads, streetlights, drainage systems, power utilities, and vegetation. When hazards emerge in these areas, the cost of delayed resolution is measured not just in money, but in public safety.

  Despite this, the dominant model for hazard resolution in most municipalities remains fundamentally manual, fragmented, and reactive. A reported hazard must pass through multiple human touchpoints before any contractor reaches the site such as classification, budget verification, fund reallocation approvals, and contractor coordination all happen as disconnected steps with no shared state.

  This introduces compounding delays at every handoff. A critical road collapse that endangers lives may sit unresolved for days because a budget-check email went unanswered, or the first contractor on a list was unavailable and no fallback logic existed.

- **Target Domain:** Civic infrastructure operations and autonomous public safety response. GridGuard targets municipal governments, city operation centres, and public works departments that manage infrastructure maintenance at scale.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Proposed Solution Summary:** GridGuard is an agentic AI system that autonomously resolves municipal hazards end-to-end. Powered by GLM-5.1 as the core intelligence layer and orchestrated by LangGraph, the system ingests a hazard report, classifies its type and risk severity, checks the relevant budget, reallocates funds autonomously if required, dispatches the nearest available contractor, and escalates to a human operator only when no viable resolution path exists.

  Every decision is logged to a full PostgreSQL audit trail. Every intermediate state is persisted in Redis so execution can resume after interruption. The entire operation from report to contractor dispatch requires zero human intervention under normal operating conditions.

2. Background & Business Objective

- **Background of the Problem:** When a city citizen encounters a public hazard like a pothole, a broken streetlight, or a flooded drain, they typically have no fast, accessible way to report it. Existing channels such as hotlines, municipal portals, and walk-in counters are slow, unfamiliar, or unknown to most residents. As a result, many hazards go unreported until they become emergencies.

On the authority side, even when reports do arrive, the resolution process follows a fragmented manual chain:

- A report arrives through an unstructured channel such as phone, email, or paper form
- A coordinator manually reviews, classifies the hazard type and urgency
- A finance officer is contacted separately to verify budget availability
- If budget is insufficient, an approval chain for fund reallocation begins
- A logistics officer manually identifies and contacts an available contractor
- If that contractor is unavailable, the entire process restarts from scratch

© UMHackathon 2026

There is no shared state between these steps. Each handoff introduces latency. There is no fallback automation when a step fails. Incidents can fall through the cracks with no escalation record.

- **Importance of Solving This Issue:**

Main Point	Impact
Manual classification	Inconsistent prioritisation; low-risk treated same as critical
Siloed budget visibility	Finance and operations work from different systems
No fund reallocation automation	Budget shortfalls require manual approval chains, causing critical delays
No contractor fallback	If primary contractor unavailable, resolution stalls completely
No real-time status visibility	Supervisors have no live view of incident progress or decisions
Silent failures	Incidents can be lost between handoffs with no audit trail

- **Strategic Fit / Impact:** GridGuard directly aligns with the hackathon requirement to place GLM on Z AI as the critical component of the solution. GLM-5.1 serves as the sole decision-making intelligence in the Reasoning node of the agentic graph. It decides structured context (hazard type, risk level, budget state) and emits a strict JSON policy decision that the workflow engine executes deterministically. This is not a peripheral integration; GLM is the decision core

© UMHackathon 2026

3. Product Purpose

3.1. Main Goal of the System: To allow any city citizen to report a municipal hazard by submitting a photo and description, and to autonomously process that report through AI-driven triage, budget assessment, and contractor dispatch that requires only a single approve or reject decision from a city operations manager,

3.2. Intended Users (Target Audience):

- Citizen
- City Operations Manager
- Contractor

4. System Functionalities

4.1. Description: GridGuard operates as a stateful agentic workflow engine. Upon receiving a hazard incident, it executes a LangGraph state machine that cycles through classification, financial assessment, AI-driven policy decisions, and logistics coordination that loops back autonomously when constraints require reallocation before proceeding.

4.2. Key Functionalities:

- **Hazard triage & classification:** Classifies incoming incidents into one of five strict hazard categories which is ROAD_PAVEMENT, UTILITY_POWER, WATER_SEWAGE, VEGETATION, LIGHTING. It also Assigns risk level (LOW, MEDIUM, HIGH, CRITICAL) based on hazard type and report data. Finally, it triggers the appropriate budget category for financial assessment
- **Autonomous Budget Check & Fund Reallocation:** Queries the DuckDB operational budget for the relevant hazard category. If the budget is sufficient, proceed to contractor dispatch. If budget is zero and risk level is HIGH or CRITICAL, GLM triggers TRANSFER_FUNDS: draws from Safety_Contingency fund and loops back to re-check budget. All fund transfer events are logged to the PostgreSQL audit trail

© UMHackathon 2026

- **Contractor Dispatch with Fallback Loop:** Sorts available contractors by distance from incident location. It dispatches the nearest available contractor. If the contractor is unavailable, retries with the next closest and continues until the contractor found or retries exhausted. On retry exhaustion, escalates to HUMAN_ESCALATION and never fails silently .
- **Real-Time Dashboard:** Live incident feed with status badges via Server-Sent Events (SSE). Decision trace view showing per-node summaries, GLM decisions, and tool calls. Budget board with live visualisation for all 5 hazard categories. Dark mode and high-contrast accessibility mode

4.3. AI Model & Prompt Design

4.3.1. Model Selection: GridGuard uses GLM-5.1 accessed through an OpenAI-compatible endpoint. GLM-5.1 was selected as the required core intelligence component for this hackathon. It serves exclusively as the Reasoning node in the LangGraph workflow, receiving structured incident context and emitting a strict policy decision JSON. Its structured output capability and reliability on constrained JSON schemas make it suited for deterministic agentic workflows.

4.3.2. **Prompting Strategy:** GridGuard uses a structured single-turn prompting strategy at each Reasoning node invocation. The prompt provides GLM with the current workflow state (hazard category, risk level, budget remaining, previous decisions) and instructs it to respond exclusively with a valid JSON object conforming to the decision schema. This is a constrained zero-shot approach: no examples are injected, but the output format is strictly defined to guarantee parseability and auditability.

4.3.3. **Context & Input Handling**: Each GLM invocation receives only the context relevant to the current incident: hazard type, risk classification, budget state, available contractors, and decision history for that incident. The context is bounded by incident scope

(thread_id = incident_id), preventing cross-incident contamination. Input size is inherently controlled by the structured state model until there is no free-text user input to the LLM.

4.3.4. **Fallback & Failure Behavior:** If GLM returns a malformed or unparseable response, the system falls back to deterministic policy logic using the same JSON schema that no hallucination can propagate into a tool call. If GLM is unavailable (no GLM_API_KEY set), the deterministic fallback is the default execution path. At the workflow level, if no viable resolution path exists after all retries, the system unconditionally transitions to HUMAN_ESCALATION, a named, logged, visible state. There are no silent failures.

5. User Stories & Use Cases

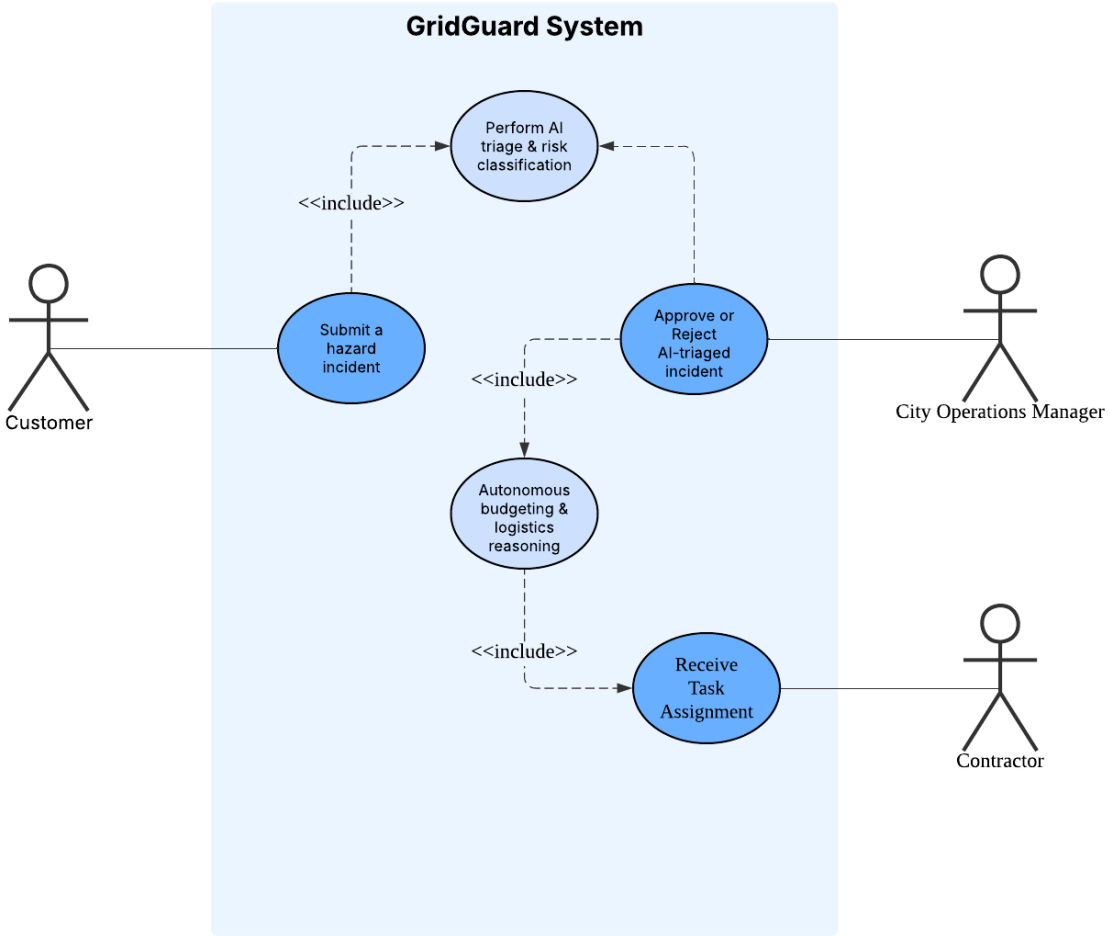
- **User Stories:** As a citizen, I want to submit a hazard incident so that the system automatically classifies, budgets, and dispatches a contractor without requiring my follow-up. As a city operations manager, I want to review an AI-triaged incident summary so that I can approve or reject it with a single action. As a contractor, I will be assigned to the incidents based on location of the incidents and availability.
- **Use Cases (Main Interactions):**

Citizen Report Flow: Citizen opens the app, uploads a photo of the hazard, enters their location and a short description, and submits. GridGuard receives the report, runs AI triage to classify the hazard type and risk level, and queues it for manager review.

Manager Approval Flow: The operations manager sees the pre-classified incident on the dashboard that shows photo, description, location, hazard type, and risk level. Manager clicks Approve or Reject. On approval, the agentic workflow triggers automatically: budget check, fund reallocation if needed, and contractor dispatch, all without further manager involvement.

© UMHackathon 2026

- **Use Case Diagram:**



© UMHackathon 2026

Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- Stateful cyclic agentic workflow with LangGraph (Triage → Financial_Check → Reasoning → Logistics_Dispatch loop)
- Autonomous budget reallocation from Safety_Contingency fund for HIGH/CRITICAL incidents
- Contractor dispatch with distance-sorted fallback loop and human escalation on exhaustion
- Full audit trail persisted in PostgreSQL for every node execution
- Operational budget tracking in DuckDB across 5 hazard categories
- Real-time incident feed and decision trace via SSE
- Live budget visualisation dashboard
- Deterministic fallback policy when GLM is unavailable
- Dark mode and high-contrast accessibility mode on the frontend

7. Features Not Included (Scope Control)

- Mobile application (web dashboard only for this MVP)
- Live payment processing or financial system integration
- Multi-city or multi-tenant deployment
- User authentication and role-based access control (deferred post-MVP)
- Push notifications or email/SMS alerting to contractors

8. Assumptions & Constraints

- **LLM Cost Constraint:** Each incident triggers one GLM invocation at the Reasoning node. Token consumption is bounded by the structured state context (estimated 200–400 tokens per call).
- **Technical Constraints:** GLM_API_KEY must be set as an environment variable before starting Docker Compose for live GLM decisions. Without GLM_API_KEY, the system

© UMHackathon 2026

operates on deterministic policy fallback only. DuckDB operates as an in-process operational store; it is not suitable for high-concurrency production workloads in its current configuration. Contractor dispatch uses mock tool interfaces; real contractor APIs are not integrated in this MVP

- **Performance Constraints:** LangGraph graph execution is synchronous per incident thread. Concurrent incidents run on separate threads. GLM response latency is dependent on the Z AI endpoint availability and load.

- **User Input:** Hazard category must conform to the strict enum: ROAD_PAVEMENT, UTILITY_POWER, WATER_SEWAGE, VEGETATION, LIGHTING. Risk level must be one of: LOW, MEDIUM, HIGH, CRITICAL. Incident inputs not conforming to the schema are rejected at the API layer before entering the workflow.

9. Risks & Questions Throughout Development

- **GLM hallucination:** Model returns malformed or policy that violates decision JSON.
- **Redis state loss:** Checkpoint lost on Redis restart, preventing incident resume.
- **Contractor exhaustion**: All contractors unavailable; incident cannot be dispatched

© UMHackathon 2026
Email: umhackathon@um.edu.my